CompSci 390-03-26S

Distributed and Networked Systems

Synchronization and Servers

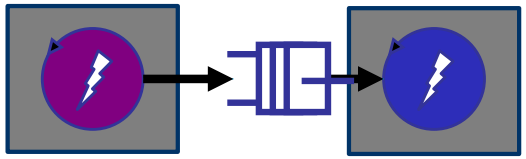
Channels and goroutines

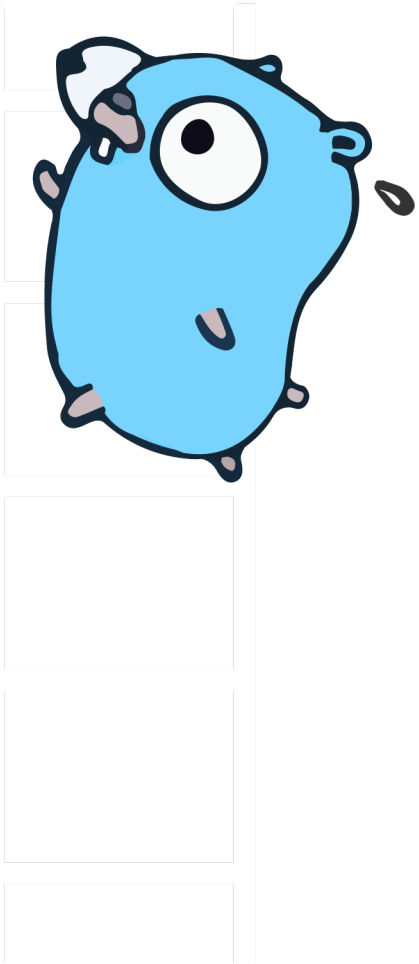
```
func main() {
        ch := make(chan int)

        go func() {
                ch <- 1
        }()

        <- ch
}
```

send

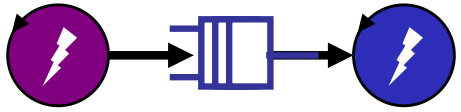




Inside a channel

- Bounded buffer/queue to hold messages in transit (PCBB).
- Compare to a **pipe** in Unix: buffer can hold a chunk of bytes.
- Go channels have (optionally) a buffer size specified at make time.
  - `reqCh := make(chan Request, 16)`
- Block on full/empty provides **flow control** to speed-match sender/receiver.
- By default, sender/receiver must alternate strictly: **ping pong**.

```
send
s.Down()
...
r.Up()
```

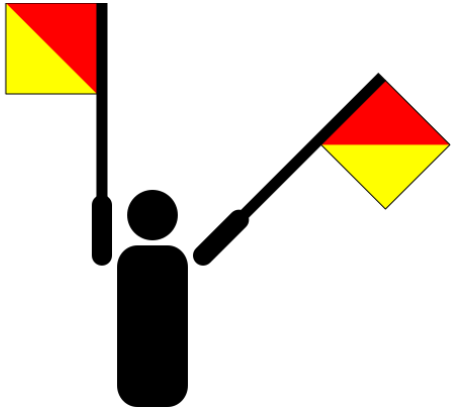


Sender Producer **Receiver Consumer**

```
receive
r.Down()
...
s.Up()
```

Semaphores

- A **semaphore** is an atomic counter:
  - Non-negative integer value
  - API: increment/decrement (V/P, Up/Down)
  - Down: **sleep** on decrement 0
  - Up: **wake** on increment 0
  - **Equivalent in power** to monitors
  - **Elegant!**



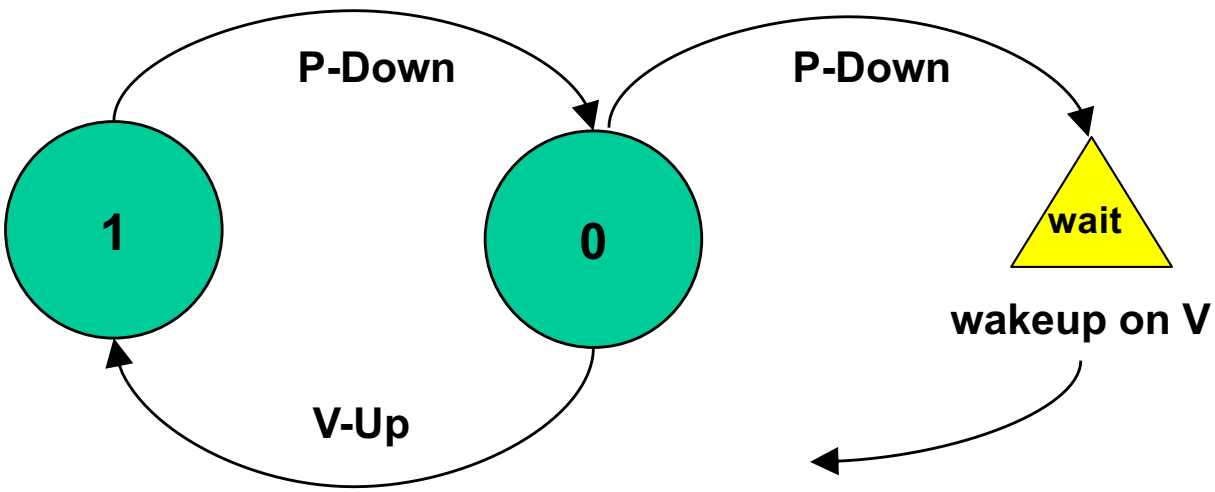
The name **semaphore** comes from a system for communicating with flags, used until the early 19th century. **Now you know.**

Curious?

http://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf

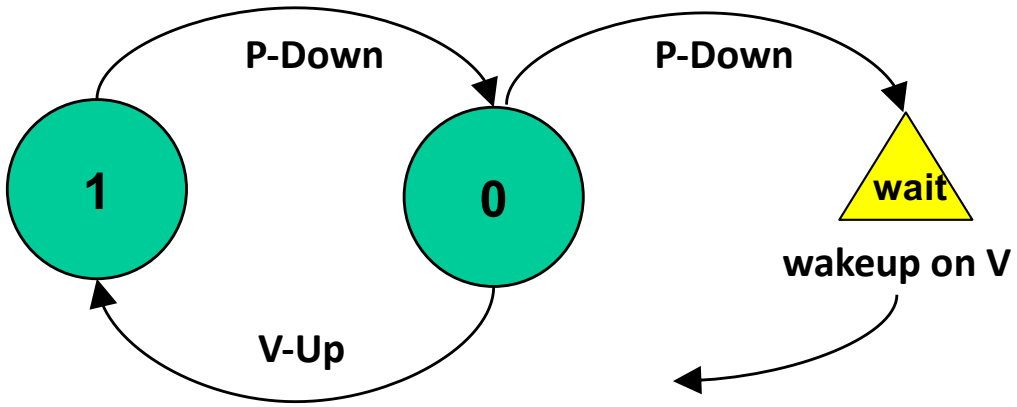
Example: binary semaphore

- A binary semaphore takes only values 0 and 1.
- It requires a usage constraint: the set of threads using the semaphore call P and V in strict alternation.



Mutual exclusion with a binary semaphore

- A **mutex** is equivalent to a binary semaphore with an initial value of 1, for which each thread calls P-V in strict pairs.
- Once a thread A completes its P, no other thread can P until A does a matching V.



```
func main() {
        ch := make(chan int)

        go func() {
                ch <- 1
        }()

        go func() {
                <-ch
        }()

        select {
        case <-ch:
                fmt.Println("R")
        case ch <- 2:
                fmt.Println("S")
        }
}
```

go-samples/deadlock.go

Select statement in Go

- For a fixed set of channel send/receive cases:
- Which case can proceed now without blocking?
  - **Receive**: input is available on channel
  - **Send**: buffer space is available on channel
- Picks one of those cases at **random**.
  - Communication proceeds
  - Executes associated statements

```
select {
case <-ch:
    fmt.Println("R")
case ch <- 2:
    fmt.Println("S")
}
```

- None can proceed? Picks **default** case, if there is one.
  - Note: defaults in loops create a “hot loop” that burns CPU.
- No case to pick? “**Park**” (sleep) thread until there is.

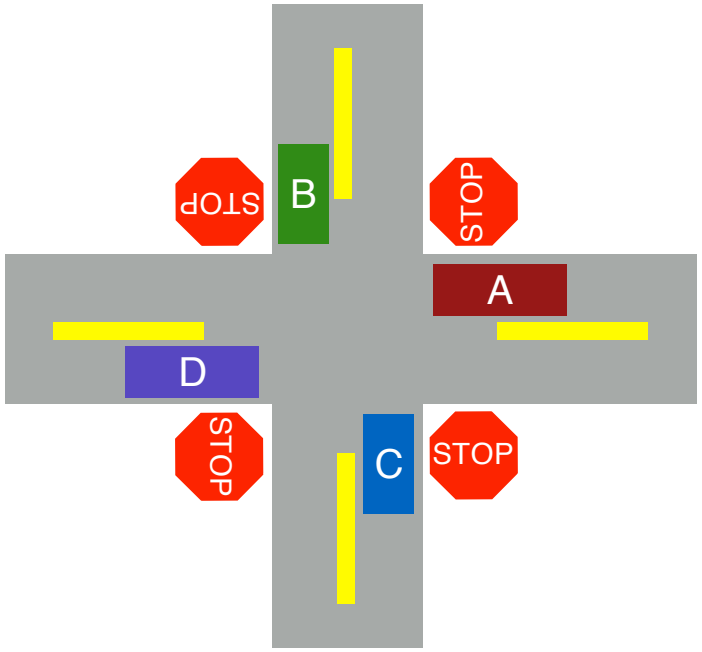
Two special cases of select

```
for {
    select {
        case c <- 0:
        case c <- 1:
    }
}
```

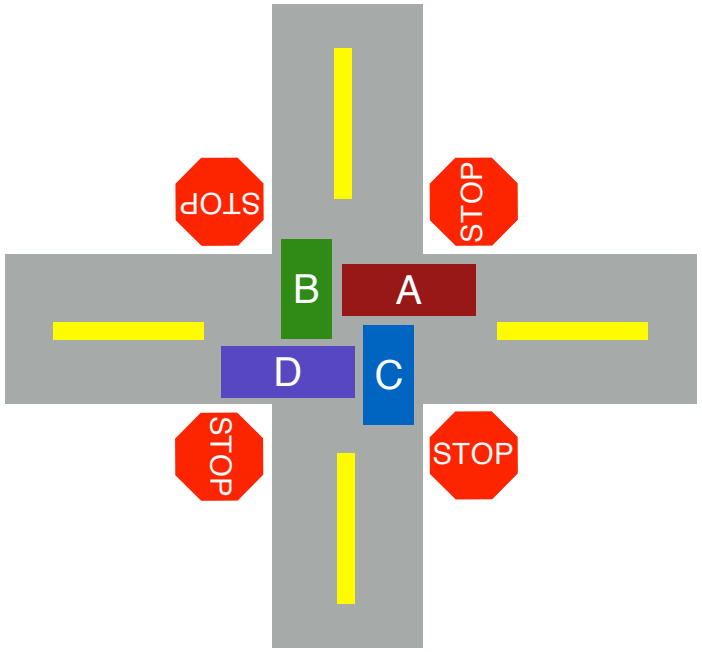
```
select {}
```

https://go.dev/ref/spec#Select_statements

Traffic – Who goes?



Traffic – Deadlock! (Assuming no reverse…)



Deadlock?

```
% go run deadlock.go
r
s
fatal error: all goroutines are asleep - deadlock!


goroutine 1 [select]:
main.main()
/Users/chase/390/go-samples/go/src/deadlock.go:58 +0xf8
exit status 2
%
```

Deadlock vs. “blocking bugs”

ACM Reference Format:
Tengfei Tu, Xiaoyu Liu, Linhai Song, and Yiying Zhang. 2019. Understanding Real-World Concurrency Bugs in Go . In *Proceedings of 2019 Architectural Support for Programming Languages and Operating Systems (ASPLOS'19).* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3297858.3304069

Bug taxonomy. We propose a new method to categorize Go concurrency bugs according to two orthogonal dimensions. The first dimension is based on the *behavior* of bugs. If one or more goroutines are unintentionally stuck in their execution and cannot move forward, we call such concurrency issues *blocking bugs*. If instead all goroutines can finish their tasks but their behaviors are not desired, we call them *non-blocking* ones. Most previous concurrency bug studies [24, 43, 45] categorize bugs into deadlock bugs and non-deadlock bugs, where deadlocks include situations where there is a circular wait across multiple threads. Our definition of blocking is broader than deadlocks and include situations where there is no circular wait but one (or more) goroutines wait for resources that no other goroutines supply. As we will show in Section 5, quite a few Go concurrency bugs are of this kind. We believe that with new programming habits and semantics with new languages like Go, we should pay more attention to these non-deadlock blocking bugs and extend the traditional concurrency bug categorization mechanism.

Application	Behavior		Cause	
	blocking	non-blocking	shared memory	message passing
Docker	21	23	28	16
Kubernetes	17	17	20	14
etcd	21	16	18	19
CockroachDB	12	16	23	5
gRPC	11	12	12	11
BoltDB	3	2	4	1
Total	85	86	105	66

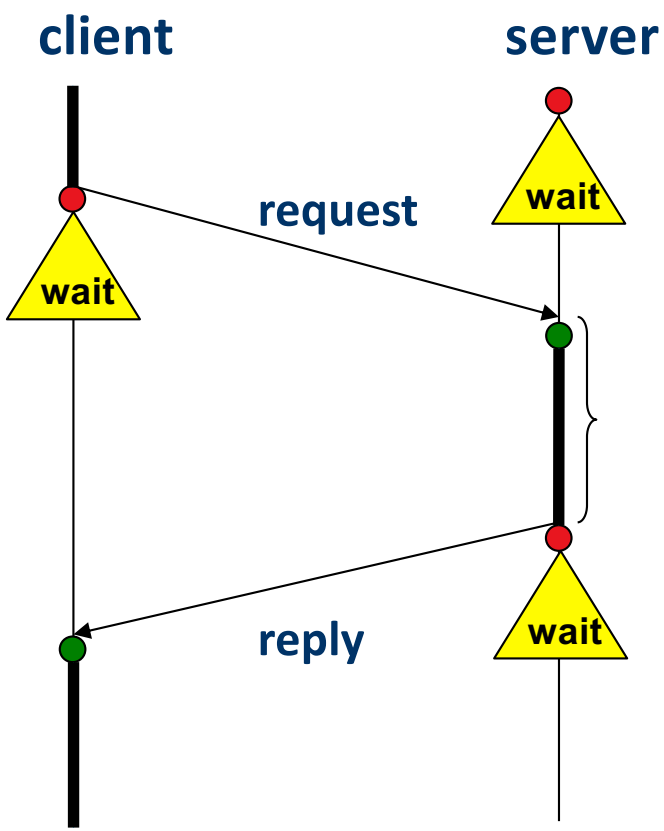
Application	Shared Memory			Message Passing		
	Mutex	RWMutex	Wait	Chan	Chan w/	Lib
Docker	9	0	3	5	2	2
Kubernetes	6	2	0	3	6	0
etcd	5	0	0	10	5	1
CockroachDB	4	3	0	5	0	0
gRPC	2	0	0	6	2	1
BoltDB	2	0	0	0	1	0
Total	28	5	3	29	16	4

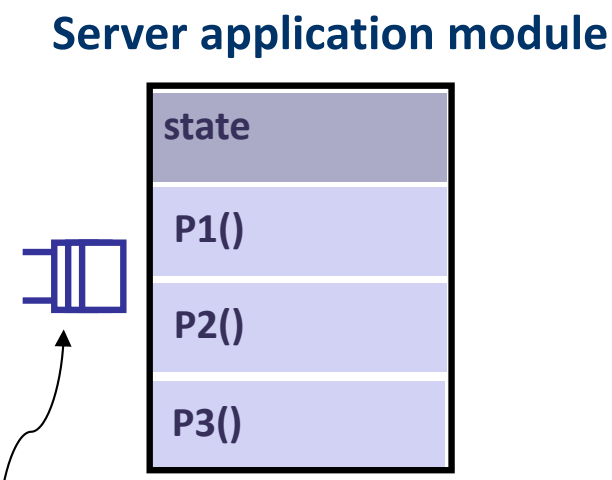
Table 6. Blocking Bug Causes. *Wait includes both the Wait function in* `Cond` *and in* `WaitGroup`*. Chan indicates channel operations and Chan w/ means channel operations with other operations. Lib stands for Go libraries related to message passing.*

Server/webapp frameworks

Borrowed bling, circa 2018 https://www.whizsolutions.co.uk

Client/server abstraction

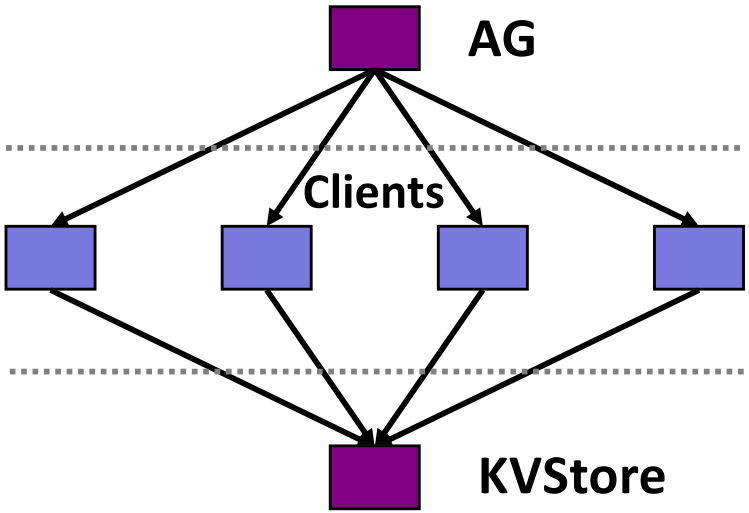




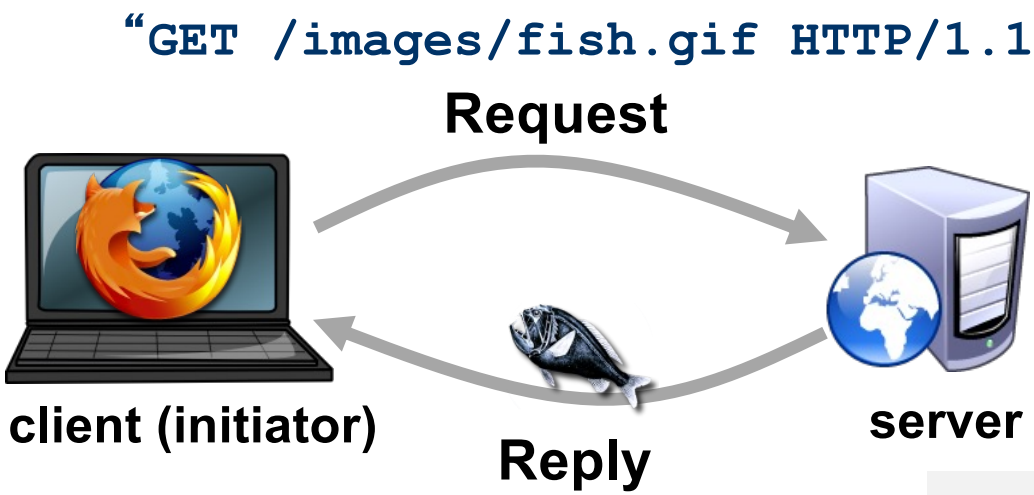
Incoming requests wait in a queue and are served as resources become available.

A distributed system in a process?

- Distributed systems are made up of processes, running on machines, communicating by sending/receiving messages over network channels.
- We can mirror their structure with actors (goroutines) sending messages over channels within a process.



Clients, servers, and the socket API




```
sd = socket(…);
connect(sd, name+port);
write(sd, request…);
read(sd, reply…);
close(sd);
```

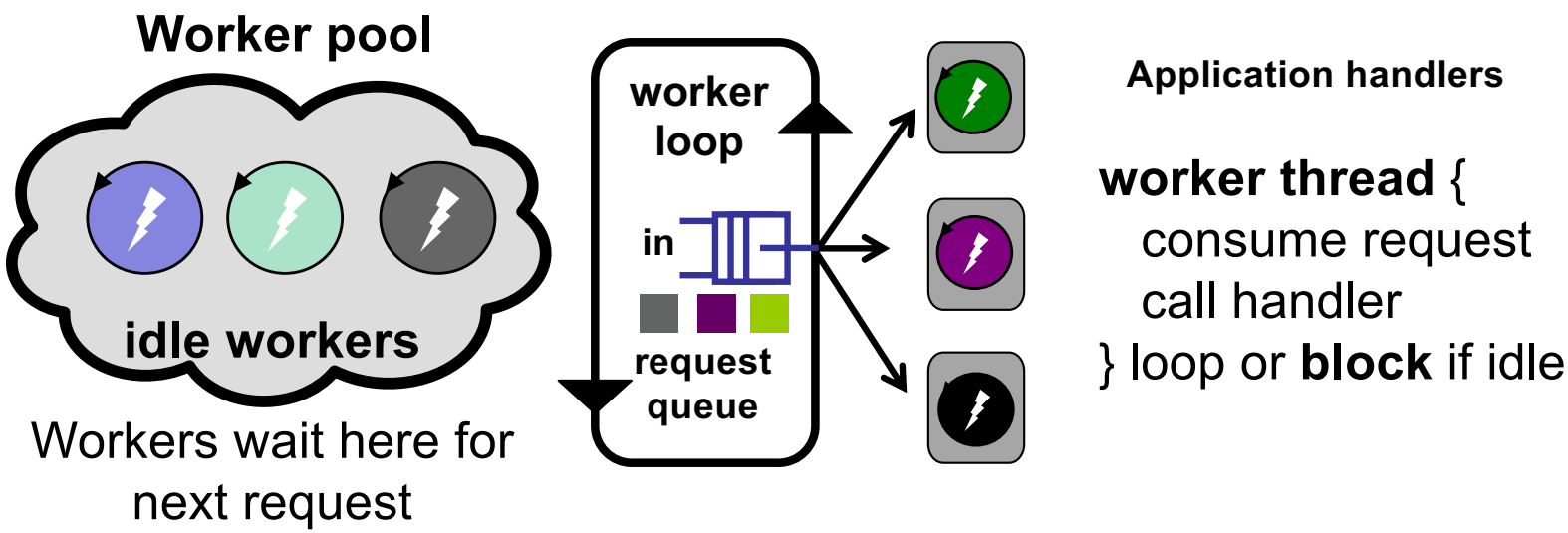
```
s = socket(…);
bind(s, name+port);
listen(s, N);
sd = accept(s);
read(sd, request…);
write(sd, reply…);
close(sd);
```

Go net package: server loop

```
ln, err := net.Listen("tcp", ":8080")
if err != nil {
      // handle error
}
for {
      conn, err := ln.Accept()
      if err != nil {
            // handle error
      }
      go handleConnection(conn)
}
```

https://pkg.go.dev/net@go1.25.6

Thread pool (WorkCrew)



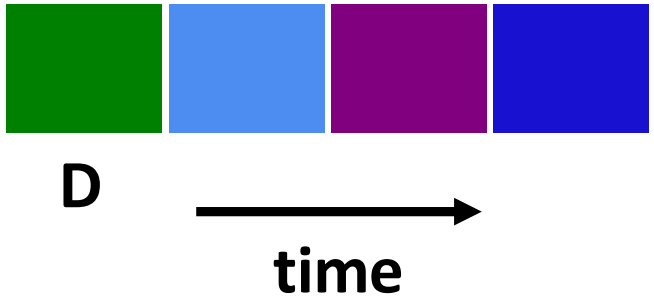
- Thread pool/WorkCrew: a design pattern for network servers.
- N workers can run in parallel on N cores.
- Queue of incoming tasks—equivalent to events+handler calls.
- Any worker thread can call any handler or run any task.

Server performance: Getting work done

- Consider a stream requests to a server.
- Performance **metrics:**
  - **Service demand**. How long does it occupy server?
  - **Response time.** How long from request to reply?
    - Service demand + waiting time (queue delay)
    - Queues everywhere!
  - **Throughput**. How many complete per unit of time?
  - **Peak** throughput: throughput at capacity.
    - **Saturation**: server capacity is fully busy.

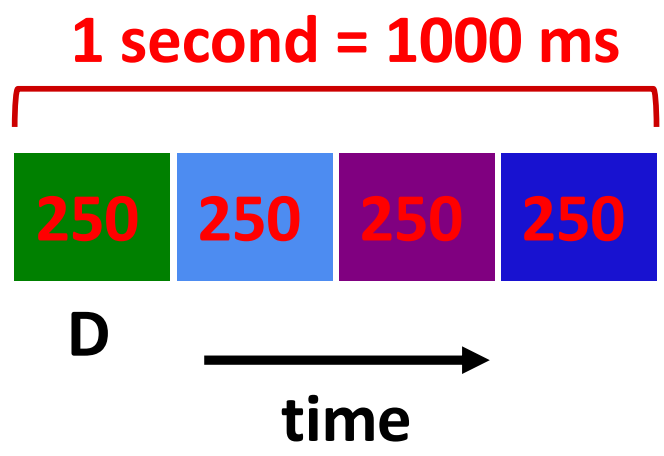
About time: intervals and rates

- Suppose requests execute **serially**: one at a time.
- Suppose each request executes for service time D (demand).
- What is the **peak throughput** X of this system?
- **Example**: D = 250 milliseconds—1/4 second



About time: intervals and rates

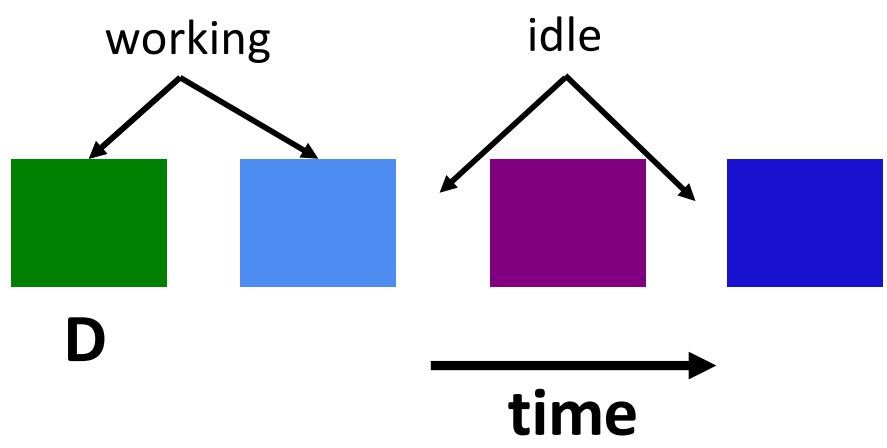
- Suppose requests execute **serially**: one at a time.
- Suppose each request executes for service time D (demand).
- What is the **peak throughput** X of this system?
- **Example**: D = 250 milliseconds—1/4 second
- X = 4 requests/second (IOPS)
- It's the **inverse**! **X = 1/D**
- (When the units match.)



Utilization

- What if requests arrive at a rate below **peak throughput** X?
- Can do work only when there is work to do!
- **Utilization U**: % of time working (a number between 0 and 1)
  - Delivered throughput = arrival rate = UX

- U=1 → resource is **saturated**.
- Cannot deliver throughput > X!
- What if requests arrive at rate **>** X?
- **Answer**: requests arriving when the resource is **busy** wait in a **queue**.
- Queues grow faster at higher U!



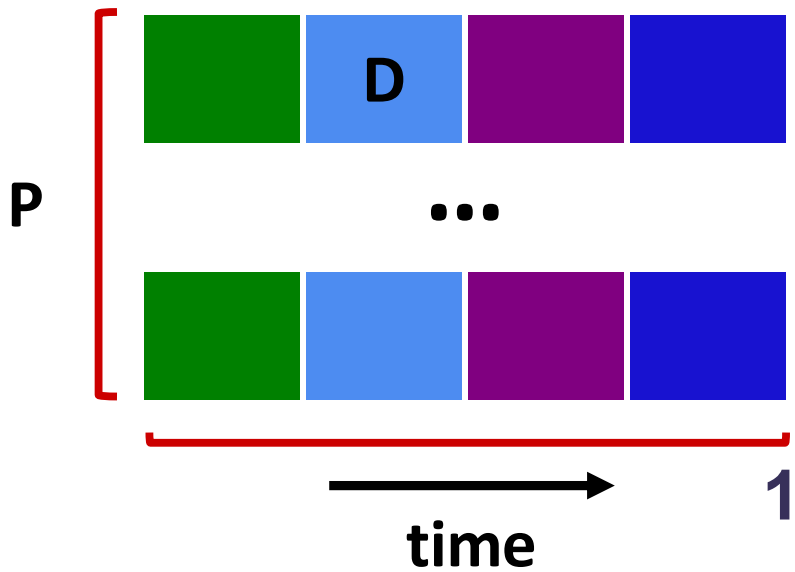
Parallelism

- Assume: requests with service demand D.
- P requests can run in parallel, e.g., on P cores or P disks.
- What is the effect of parallelism on the peak throughput X?

X = 1/D

On one core or device

X=? on P cores/devices?

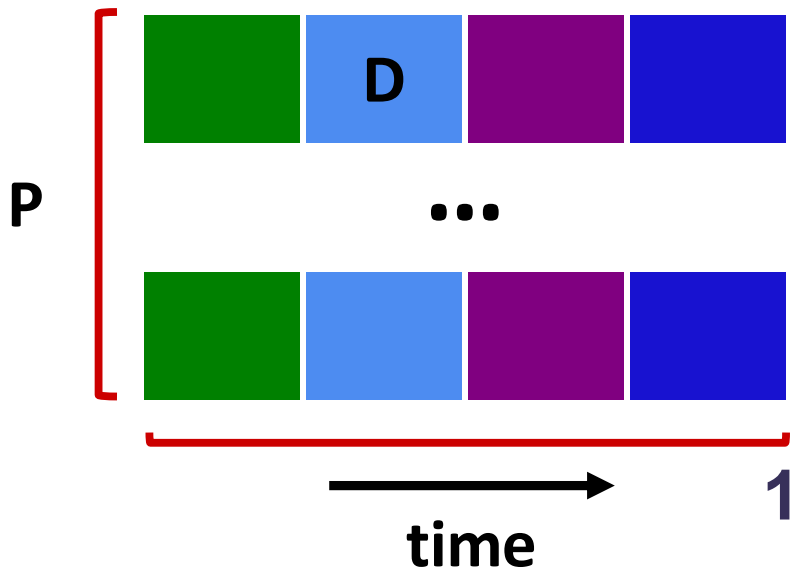


Parallelism

- Assume: requests with service demand D.
- P requests can run in parallel, e.g., on P cores or P disks.
- What is the effect of parallelism on the peak throughput X?

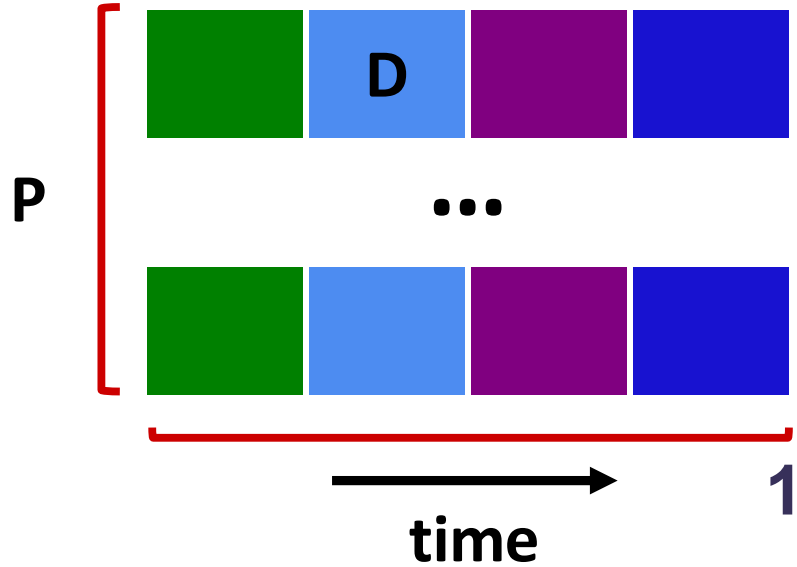
X = P/D

Increase by factor of P



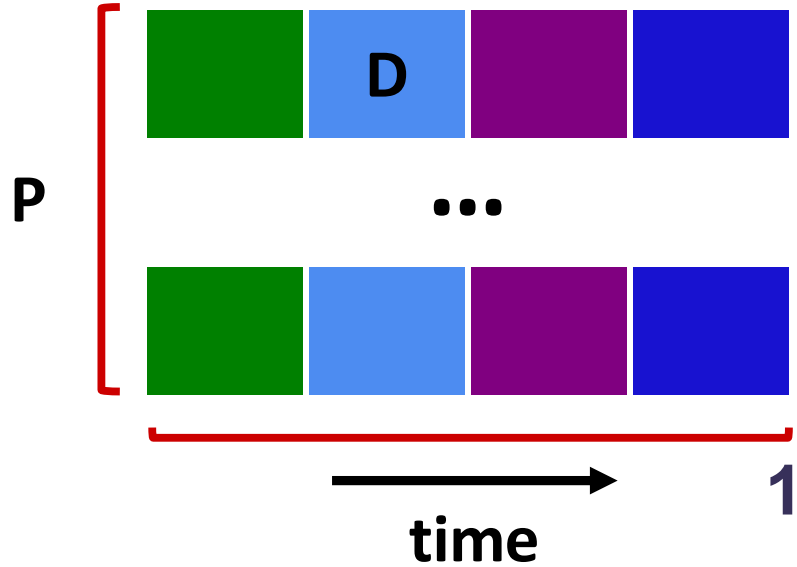
Parallelism and latency / response time

- Assume: requests with service demand D.
- P requests can execute in parallel—**serial** means P=1.
- What is the effect of P on latency or **response time R**?
  - R = from submit time to completion
  - R > D!
  - But requests could wait in a queue
  - R = D + queue delay



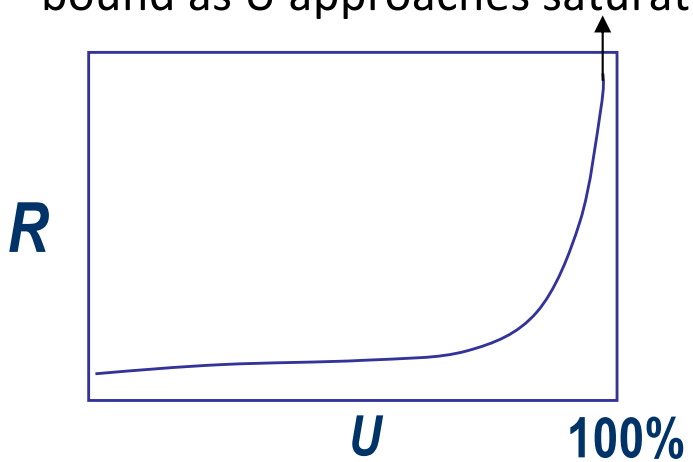
Parallelism and latency / response time

- Assume: requests with service demand D.
- P requests can execute in parallel—**serial** means N=1.
- What is the effect of P on latency or **response time R**?
- **No effect?** Still R>D
  - Parallelism "doesn't help"
  - **But**: parallelism expands capacity
  - Which reduces queue delays

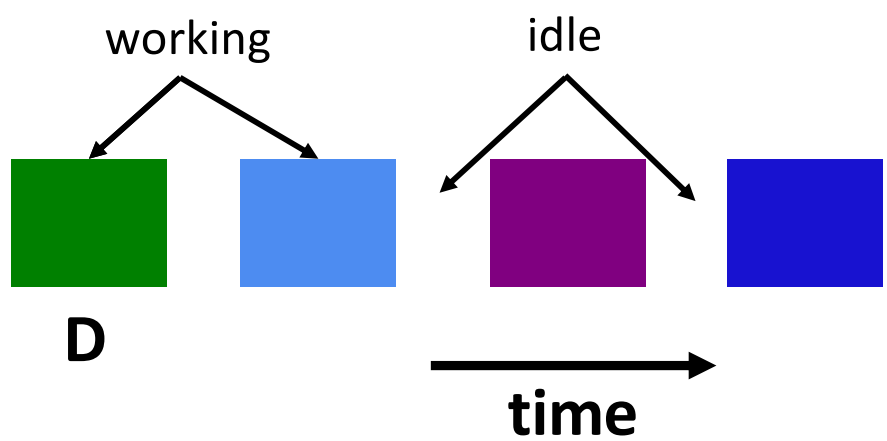


Queues and response time (for serial requests)

- **Idle time**: 1-U
- "Busyness" increases queue delays proportionally to 1/(1-U).
- It turns out average response time trends to R = D/(1-U).
- → Queues and delays grow without bound as U approaches saturation.

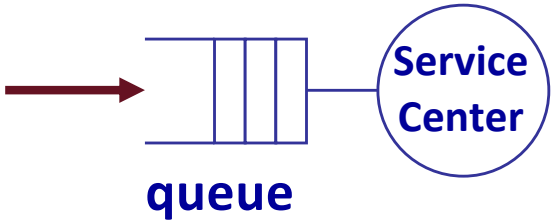


R = D(N+1)
Queue length N grows with U. If queue is FIFO, we need DN time on average to clear the queue, + time D for this new request.



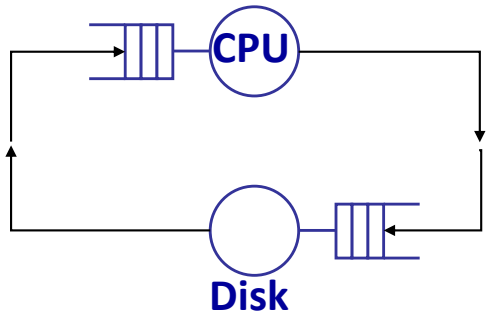
Understanding performance: queues

offered load
request stream @
arrival rate λ



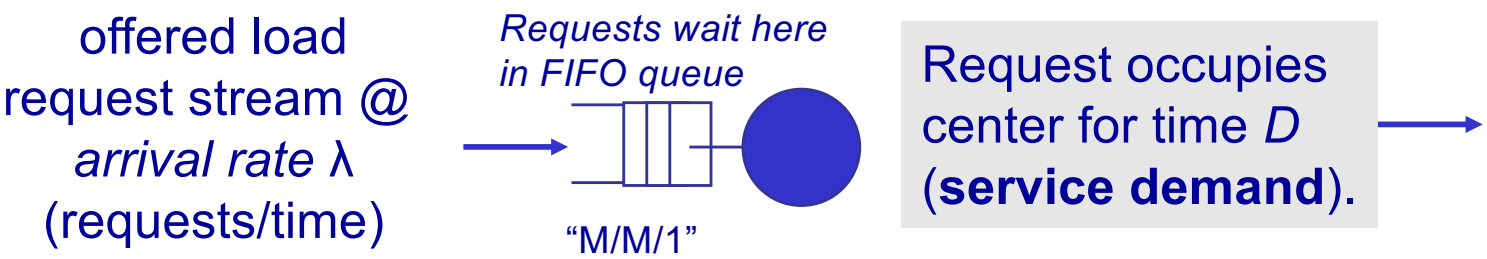
Handle request: **occupies** center for time *D* (service demand).

Systems are networks of centers and queues. To maximize overall utilization/throughput, we must think about how centers interact.



But we can also "squint" and think of the entire server as a single queuing center, and we won't go too far astray.

Queuing theory for busy people



- Consider: single service center (e.g., a core) with FIFO queue.
  - Independent request arrivals at mean rate λ (poisson arrivals).
  - i.e., Assume inter-arrival time (1/λ) and service demand (D) are exponentially distributed around their means. Noted as "M": M/M/1 queue.
  - For c service centers with c>1: M/M/c queue model, e.g., multithreaded server
- These assumptions are rarely exactly true for real systems, but they give a rough ("back of napkin") understanding of queue behavior.

Throughput

How many requests per second can a server handle?
What does it depend on?

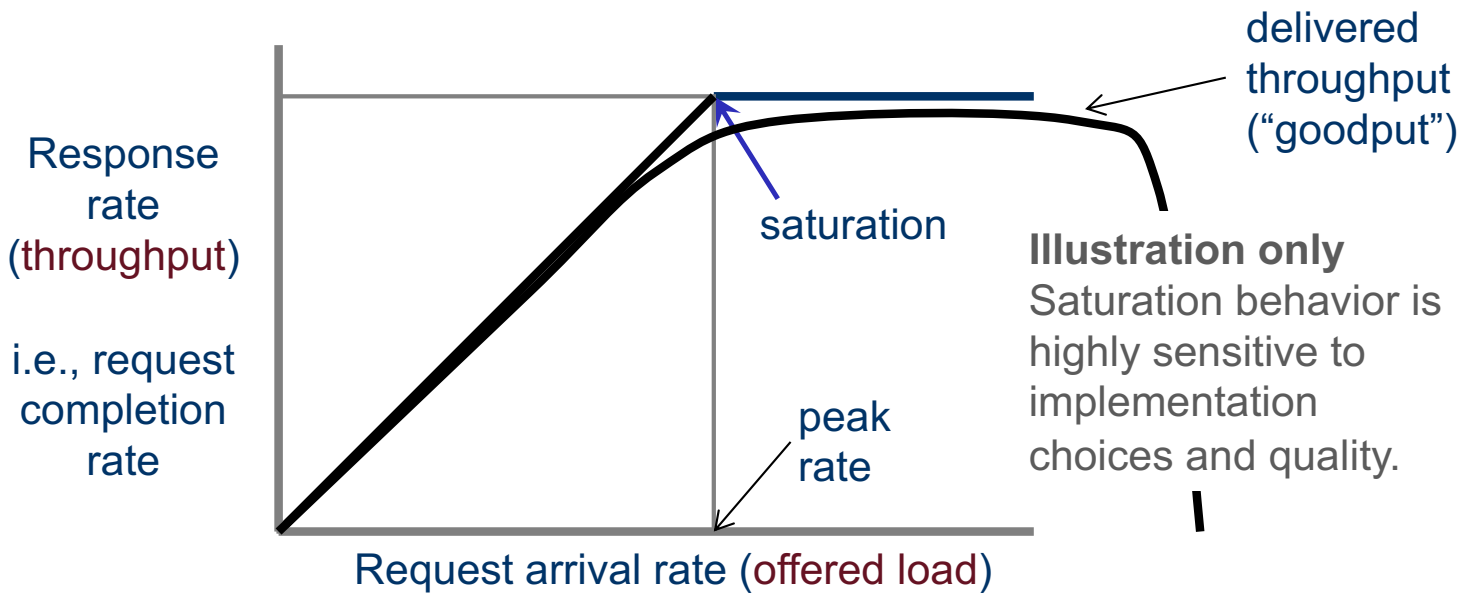
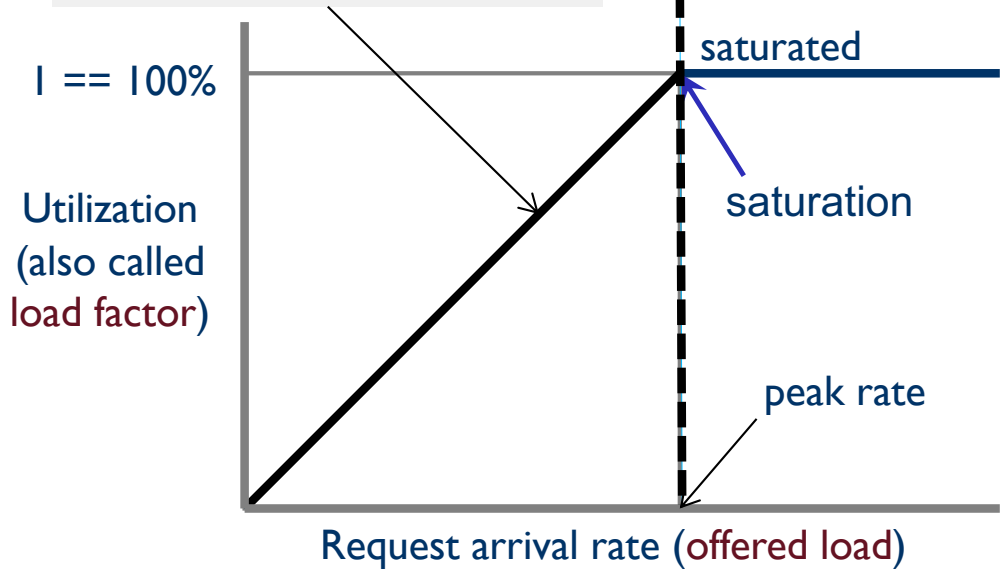


Illustration only
Saturation behavior is highly sensitive to implementation choices and quality.

Utilization

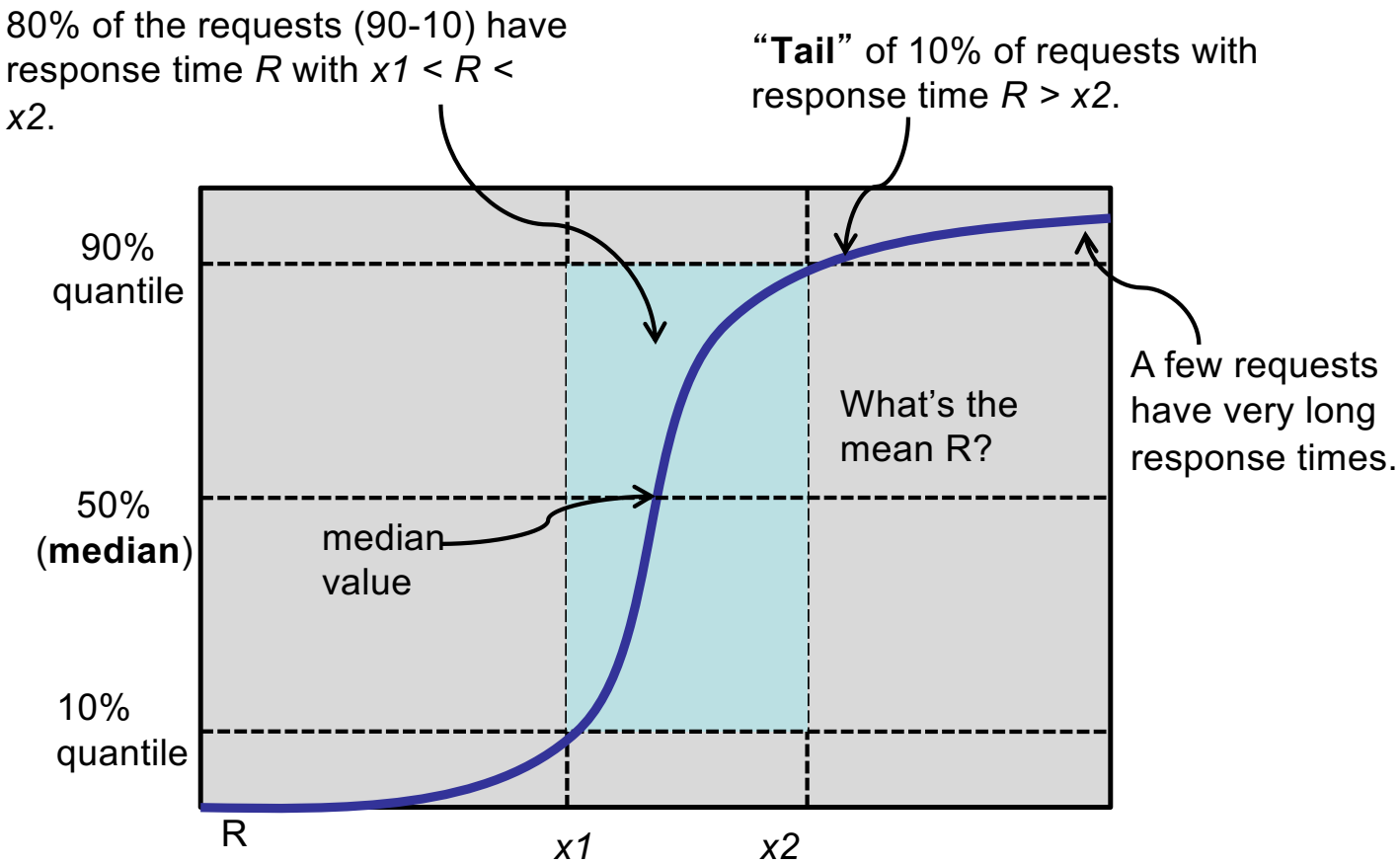
U = XD
X = throughput
D = service demand, i.e., how much time/work to complete each request (on average).

U = 1 = 100%
The server is saturated. It has no spare capacity. It is busy all the time.



This graph shows utilization (e.g., of a server) as a function of offered load. It is idealized: each request works for D time units on a single service center (e.g., a single CPU core).

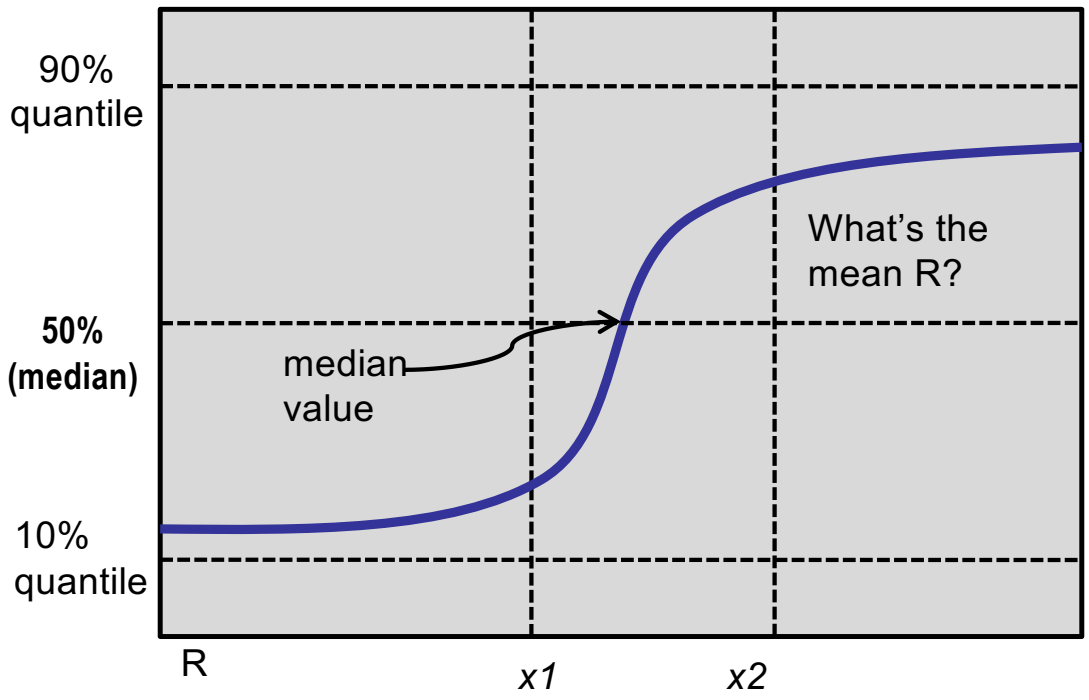
Mind the quantiles! (CDF)



<u>Understand</u> how/why the mean (average) response time can be misleading.

Tails

Better? Worse?

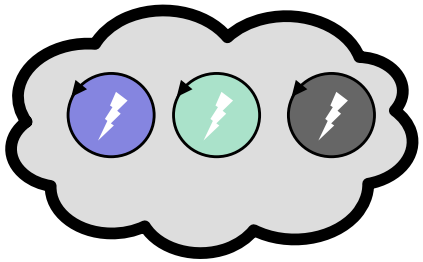


Managing load and concurrency

- **How many worker threads?**
- What if requests/tasks block?

  >N workers to keep N cores busy
- **What if request queue is full?**
  - More threads?
  - Flow control in the network?
  - Drop and say “try again later”?
  - Get a bigger server?
  - Shed load to another server?

  …



Magic elastic worker pool